

Les bases de données

Qu'est ce que c'est une BDD?

- Permet d'organiser des informations
 - Identifiants d'utilisateurs (nom, Mot2Passe...)
 - Articles (Blogs, journaux)
 - Produits (sites de vente)
- Une base de données peut être vue comme un ensemble de tableaux Excel :
 - Chaque fichier Excel est une **table**
 - Une ligne d'une **table** est un **enregistrement** (ou **tuple**)
 - Une colonne d'une table est un **attribut**

Donc c'est quoi ?

- Une base de données, c'est un ensemble de **tables**, et chaque table contient de l'information stocké sous forme de **tuple**.
- Exemple : base de données d'une école:
 - Une table **eleve**
 - Une table **classe**
 - Une table **matiere**
 - Une table **note**

Exemple 1

Eleve

Id	Nom	Prenom	Adresse	Telephone
1	Toto	Tata	3 rue Paul	1932332332
2	Pouet	Poupou	10 avenue Beaumont	1234567890
3	Besancenot	Fred	8 avenue du Prado	0987654321
4	Mistral	Jean	5 rue de l'arche	6785437584

Matiere

Id	Nom	Enseignant
1	Informatique	Lelore
2	Géographie	Sacri

Note

IdEleve	IdMatiere	Note
1	1	15
2	1	5
3	1	10
4	1	8

Règles et conseils

- Pour bien faire :
 - Utiliser une clé numérique (**un identifiant**)
 - Ne pas hésiter à découper une table en deux :
 - Dans l'exemple suivant, le modèle détermine la marque et la puissance. On peut alors créer deux tables :

Voiture				
IdVoiture	Marque	Modele	Puissance	Couleur
1	Renault	R5	5	Rouge
2	Renault	R5	5	Verte

Voiture		
IdVoiture	IdModele	Couleur
1	1	Rouge
2	1	Verte

Modele			
IdModele	Modele	Marque	Puissance
1	R5	Renault	5
2	Picasso	Citroen	12

Règle pour découper

1. Transformer chaque attribut en attribut atomique :

Quand un **attribut** contient plusieurs informations (nom et prénom, ou adresse et code postal...)

 Créer deux (ou plus) **attributs**.

2. Déterminer quelles dépendances existent entre les attributs pour chaque table

Quand un attribut détermine d'autres **attributs** (une note et un nom de matière, ou un modèle de voiture et une marque)

Créer deux (ou plus) **tables**.



Schéma relationnel

- Une fois qu'on a décidé de quelles tables faire, il est possible de faire un schéma relationnel :
 - Comme UML, mais avec :
 - À la place **des classes**, c'est **les tables**
 - À la place **des méthodes**, c'est **les attributs**
 - Il n'existe que des liens simples entre les tables
 - Penser aussi aux cardinalités



Construction d'une Base de Données



Les principes de bases de Merise

Introduction

- La phase de conception nécessite des méthodes permettant de mettre en place un modèle sur lequel on va s'appuyer.
- La modélisation consiste à créer une représentation virtuelle d'une réalité de telle façon à faire ressortir les points auxquels on s'intéresse.
- Ce type de méthode est appelé analyse.
- Il existe plusieurs méthodes d'analyse, la méthode la plus utilisée est la méthode MERISE.
- **M**éthode d'**É**tude et de **R**éalisation **I**nformatique pour les **S**ystèmes d'**E**ntreprise

Introduction

- Le but de cette méthode est d'arriver à concevoir un système d'information.
- La méthode MERISE est basée sur la séparation des données et des traitements à effectuer en plusieurs modèles conceptuels et physiques.
- La séparation des données et des traitements assure une continuité au modèle.

Le Cycle d'Abstraction

- **Le niveau conceptuel**
 - *Quelles données ?*
 - *Pour quoi faire ?*
 - *Quelles règles de gestion ?*
- **Le niveau organisationnel et logique**
 - *Qui fait quoi ? (les acteurs du système)*
 - *Quand ? (notion de temps)*
 - *Où ? (notion de lieu) et Quel type de traitement ?*
- **Le niveau opérationnel (physique)**
 - *Les techniques d'implantation du système d'information (Les matériels et les logiciels de base nécessaires, quel type de réseau)*

Les niveaux du cycle d'Abstraction

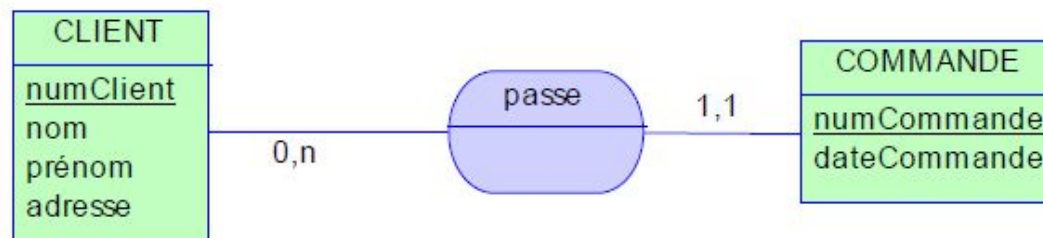
Niveaux de description		DONNEES
CONCEPTUEL		MODELE CONCEPTUEL DE DONNEES M.C.D
LOGIQUE		MODELE LOGIQUE DE DONNEES M.L.D
PHYSIQUE		MODELE PHYSIQUE DE DONNEES M.P.D

Construction d'une Base de Données

- 1^{ère} étape : MCD
- 2^{ème} étape : MLD
- 3^{ème} étape : MPD
- 4^{ème} étape : Script de la base de données

Un modèle conceptuel des données (MCD)

- Le Modèle Conceptuel de Données ou MCD est **une représentation graphique** des données, il est le plus connu et le plus utilisé des éléments de la méthode Merise. Il permet une représentation claire des données et des liens entre les données du domaine étudié.



MCD

Etapes du MCD:

- **1^{ère} étape** : recueil des informations
- **2^{ème} étape** : le Dictionnaire des Données
- **3^{ème} étape** : le Graphe de Dépendances Fonctionnelles
- **4^{ème} étape** : Normalisation du MCD

MCD

Dictionnaire de données

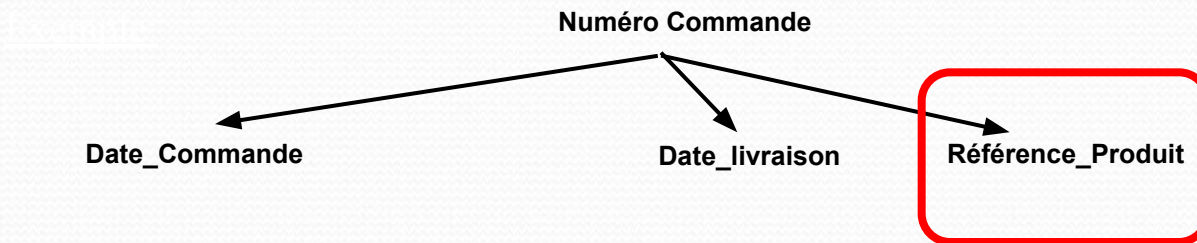
- Cette étape consiste à référencer toutes les données du domaine.

VARIABLE	SIGNIFICATION	TYPE	LG	NAT	REMARQUES
Num_Commande	Numéro de la Commande	N		PK	>=1 et <=99
Date_Commande	Date de la commande	D			
Date_livraison	Date de la livraison	D			
Référence_Produit	Référence du produit	S		PK	
Prix_Produit	Le prix unitaire du produit	N			
Quantité commandée	La quantité restante en stock	N			
...					

MCD

Graphe de dépendances fonctionnelles

- Établir une DEPENDANCE FONCTIONNELLE (DF), c'est le fait de relier de manière unique **une propriété** ou **un ensemble de propriétés** à **d'autres propriétés**.
- Ce lien est représenté par une flèche dirigée vers la droite.



MCD

Normalisation du MCD

- ▣ **1^{ère} Forme Normale (1FN)** : élémentarité des attributs et existence de l'identifiant.

Tous les attributs doivent être élémentaires par rapport au choix de gestion et il doit y avoir un identifiant à chaque entité.

ETUDIANT
Nom Prénom
Age Adresse

n'est pas 1FN

ET UDIA NT
N° étudiant
Nom Prénom
Age

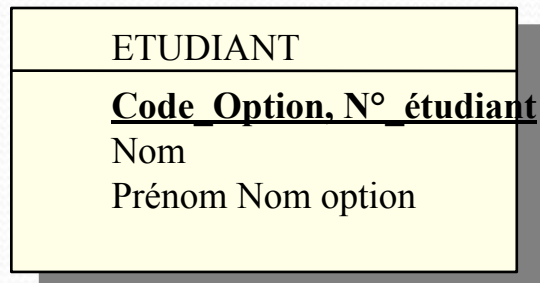
est 1FN

MCD

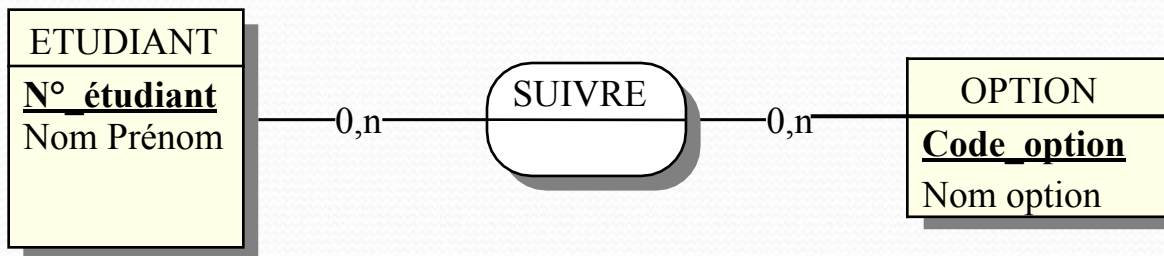
Normalisation du MCD

- ▣ **2^{ème} Forme Normale (2FN)** : Dépendance fonctionnel élémentaire de l'identifiant.

Tout attribut d'une entité doit dépendre de l'identifiant de cette entité par une dépendance fonctionnelle élémentaire.



n'est pas 2FN

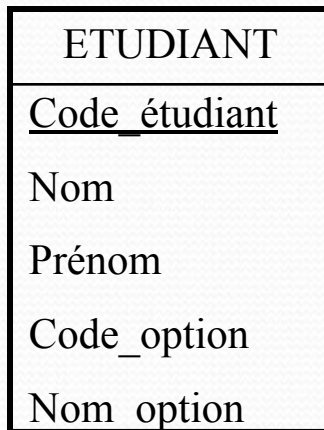


est 2FN

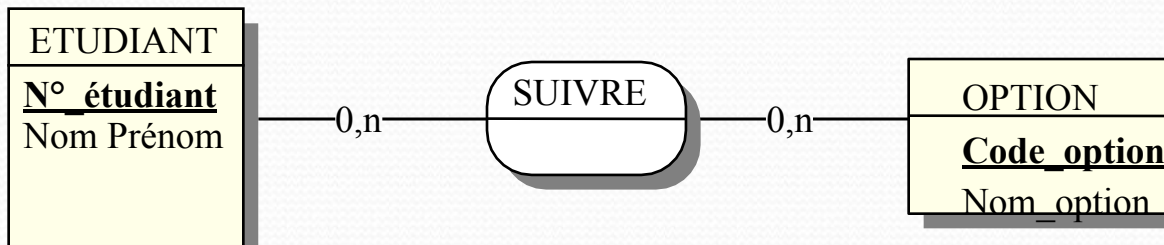
MCD

Normalisation du MCD

- ▣ **3^{ème} Forme Normale (3FN)** : Dépendance fonctionnel élémentaire et directe. Tout attribut d'une entité doit dépendre de l'identifiant par une DF élémentaire et directe.



n'est pas 3FN



est 3FN

MCD

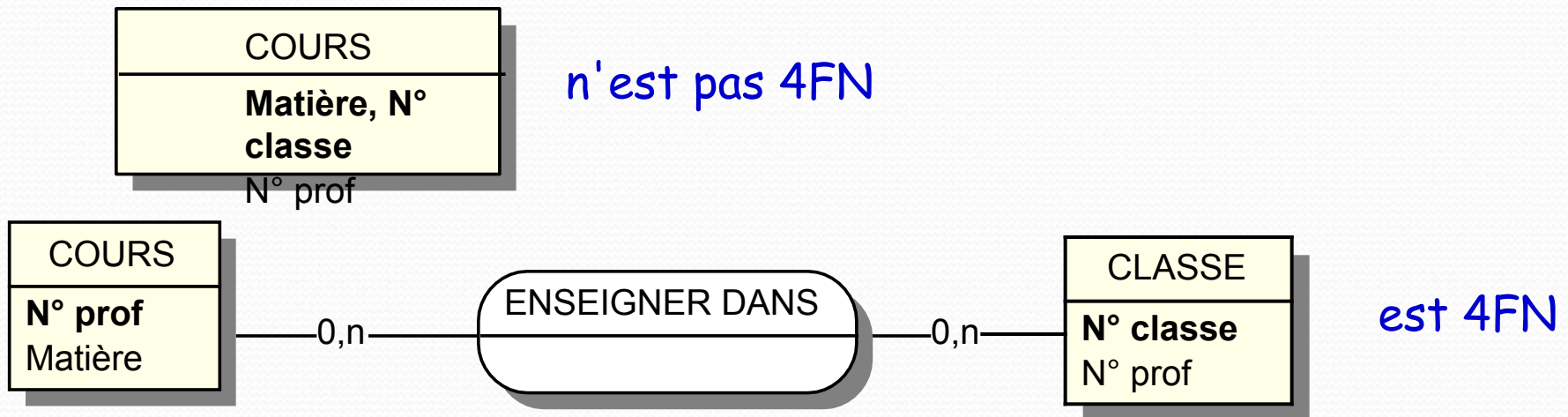
Normalisation du MCD

- ▣ **4^{ème} Forme Normale (4FN)** : DF complète (cas de l'identifiant concaténé).

Si une entité a un identifiant concaténé, un des attributs composant l'identifiant ne doit pas dépendre d'un autre attribut.

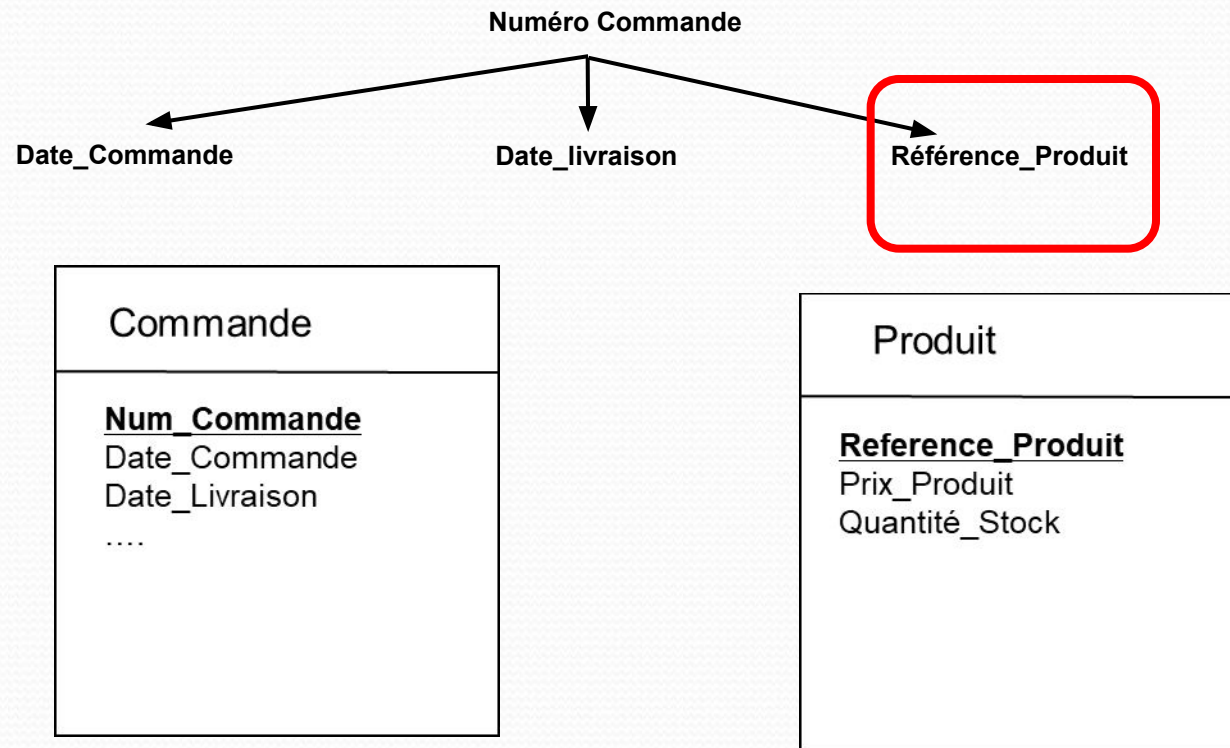
Exemple :

RG1 : tout prof enseigne une et une seule matière. RG2 : toute classe n'a qu'un seul prof par matière.



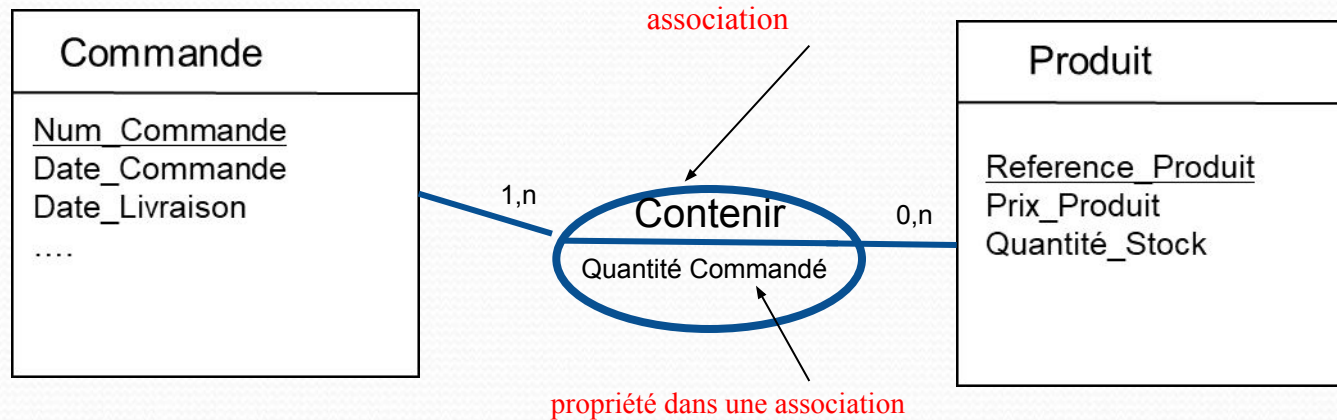
Règle N1:

- Une propriété **non identifiant** d'une entité (**table**) doit dépendre fonctionnellement de l'**identifiant**

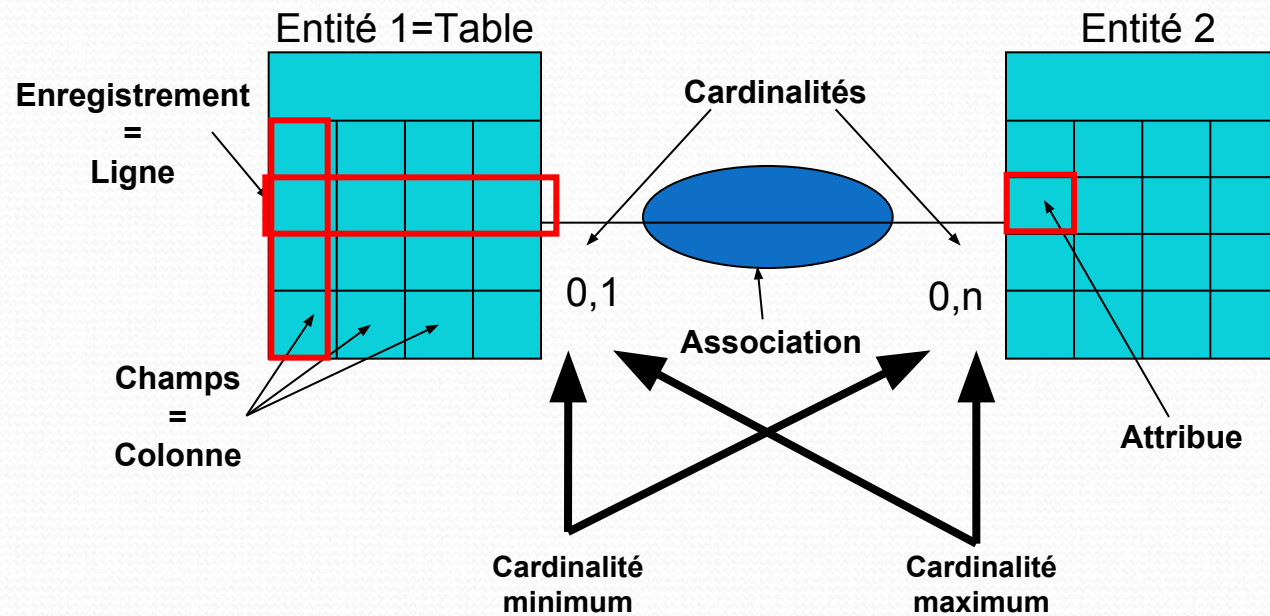


Règle N2:

- Une propriété dans une association **DOIT** dépendre fonctionnellement des **identifiants des entités** qui participent à la relation



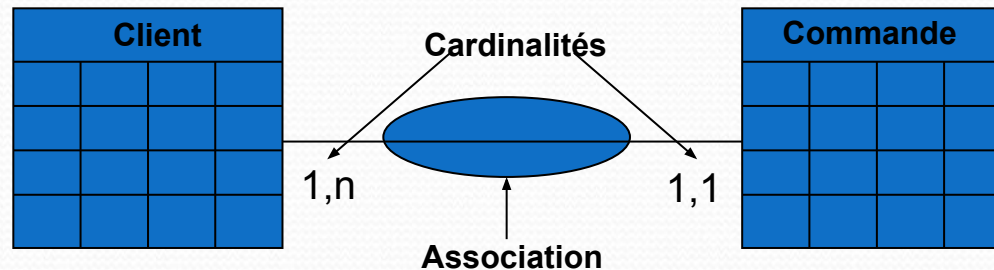
MCD



MCD

les cardinalités

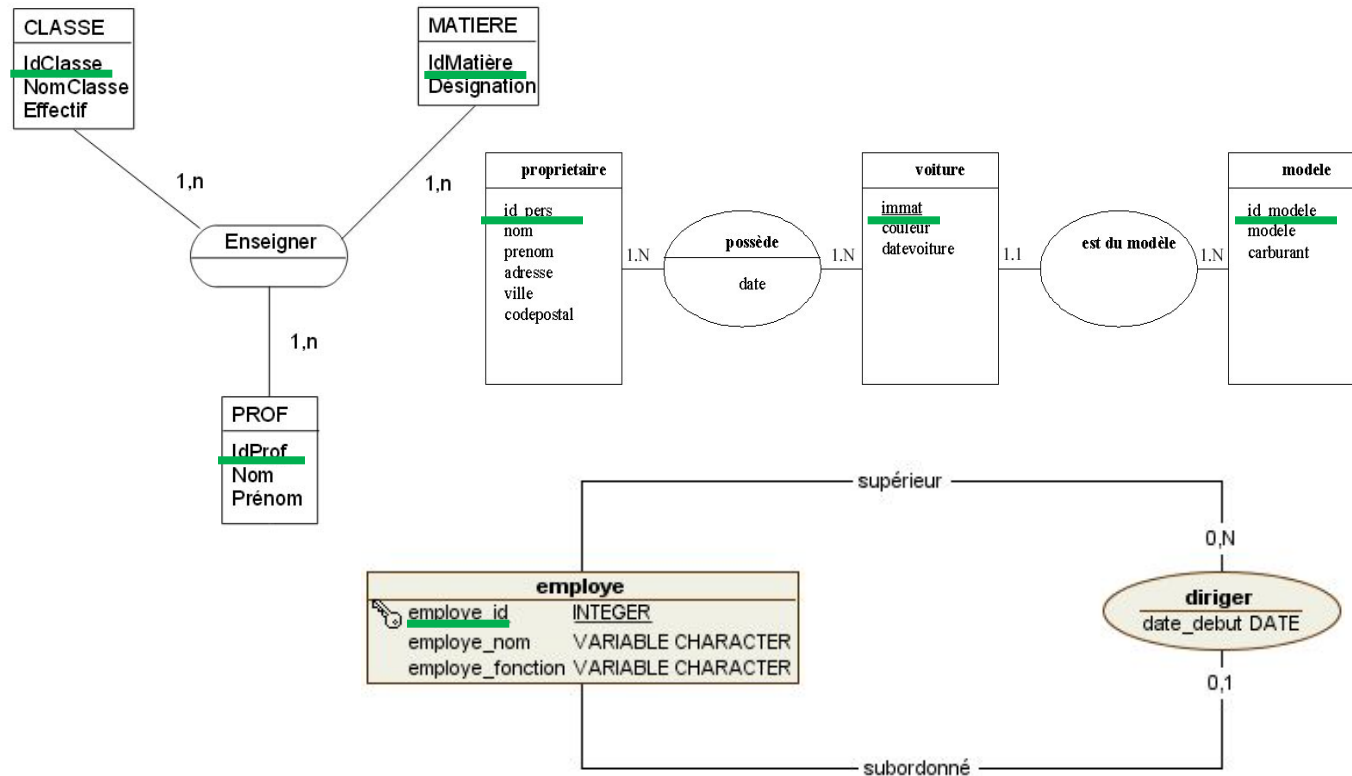
- Possibilité : 0:1, 1:1, 0:n, 1:n



Les cardinalités expriment le nombre minimum et maximum de fois qu'une entité est concernée par la relation.

MCD

Exemples de type d'associations



Erreur à évitée

- Deux **entités** d'une même **base de données** ne peuvent pas avoir le **même nom**.
- Deux **attributs** de la même **entité** ne peuvent pas avoir le **même nom**.
- Un **même attribut** ne peut être **présent** dans **plusieurs entités**.

Pour une situation donnée, il n'existe pas une «solution» unique. Le « bon modèle » est celui qui est accepté par les personnes concernées par le projet.



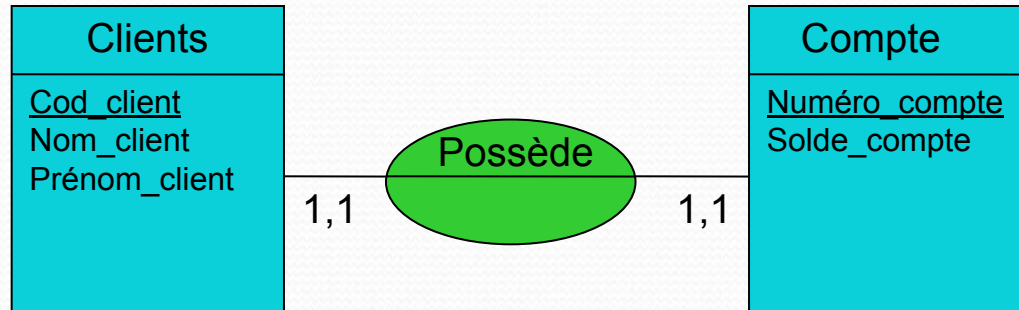
Passage du MCD à MLD

MLD

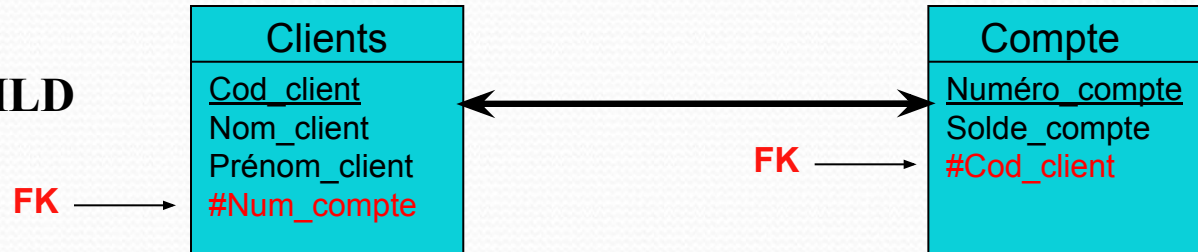
- Entités □ Tables
- Propriétés □ Champs, Attributs
- Identifiants □ Clé primaire ou Clé étrangère
- Les associations disparaissent ou deviennent des tables.
- Les cardinalités disparaissent.

Liaison de 1,1 - 1,1

MCD



MLD

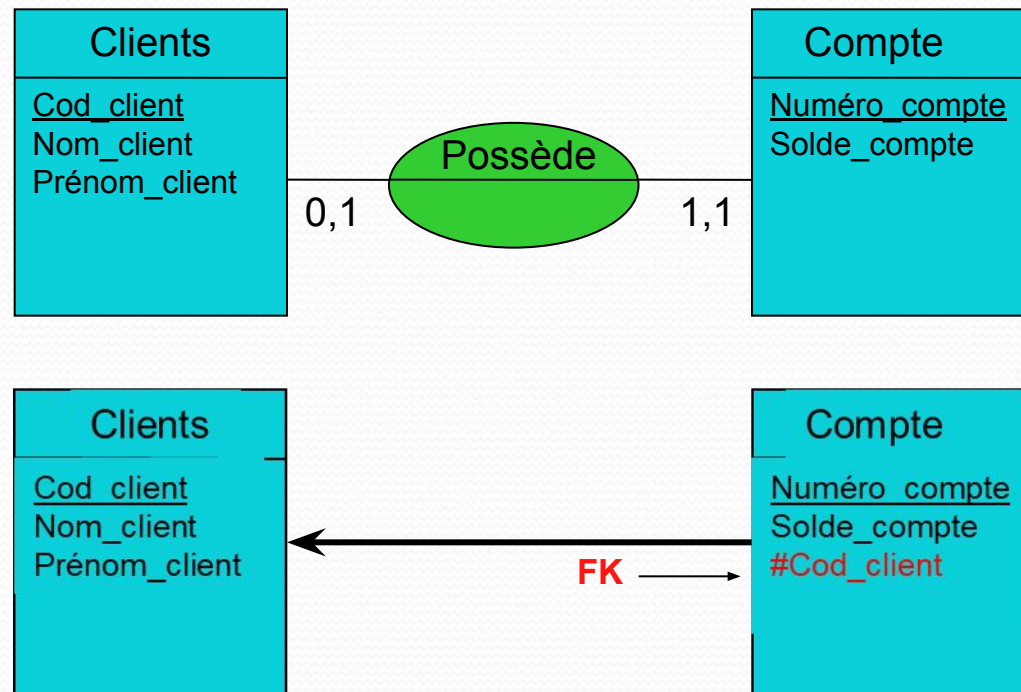


Exercice de 0,1 à 1,1

MCD

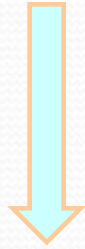


MLD

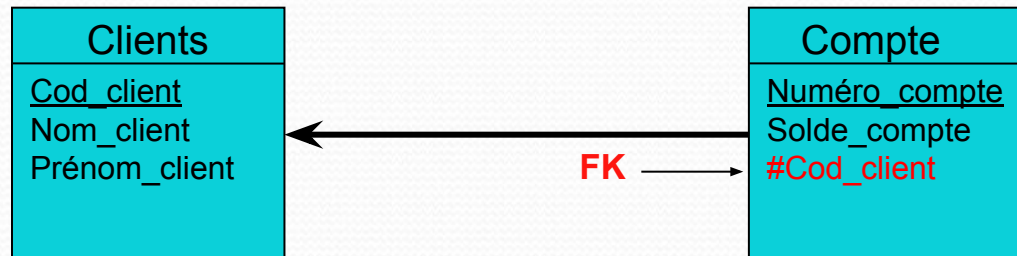
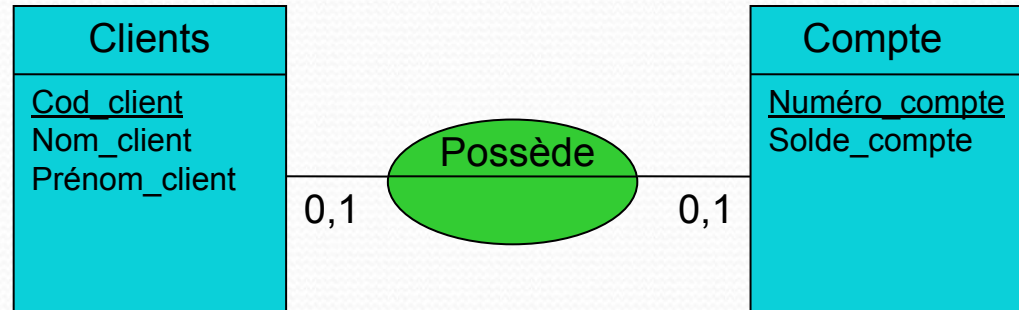


Union de 0,1 - 0,1

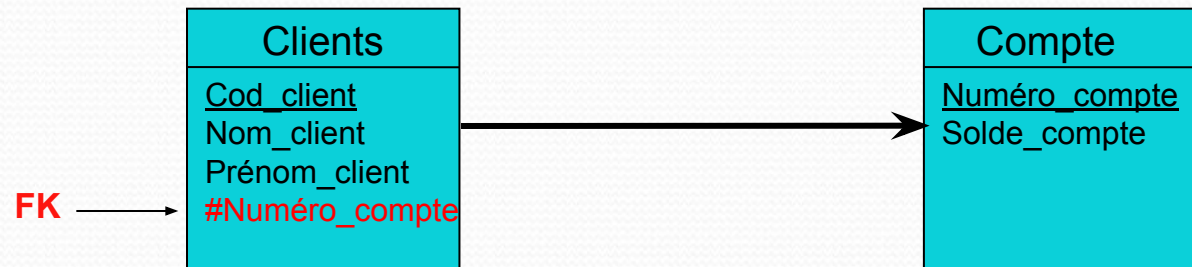
MCD



MLD

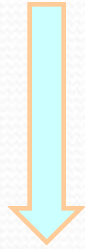


OU

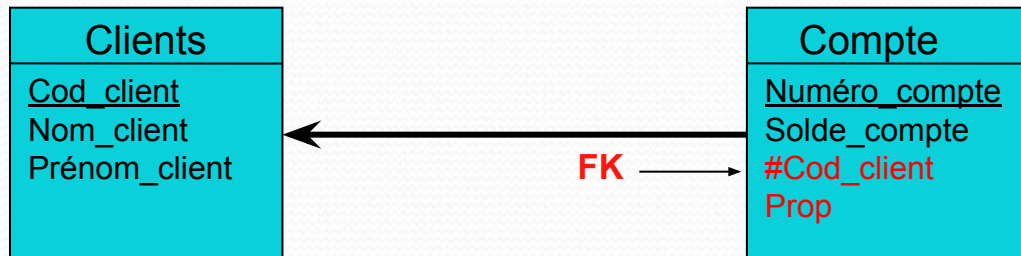
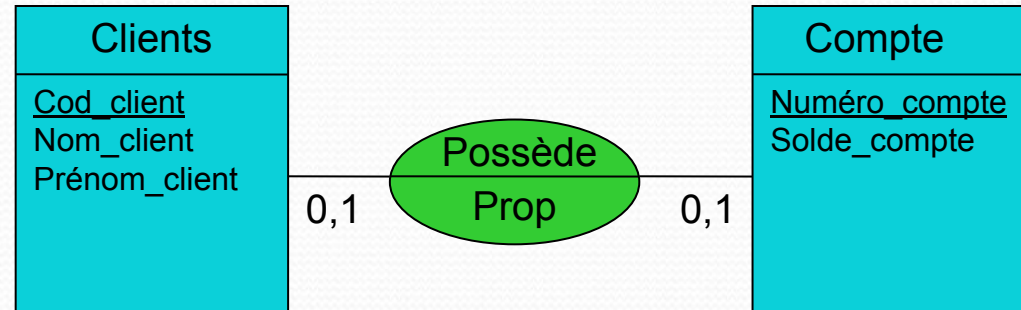


Union de 0,1 - 0,1

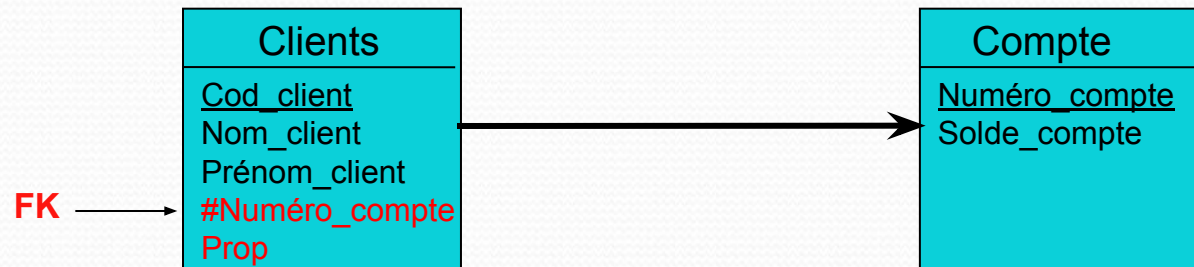
MCD



MLD



OU

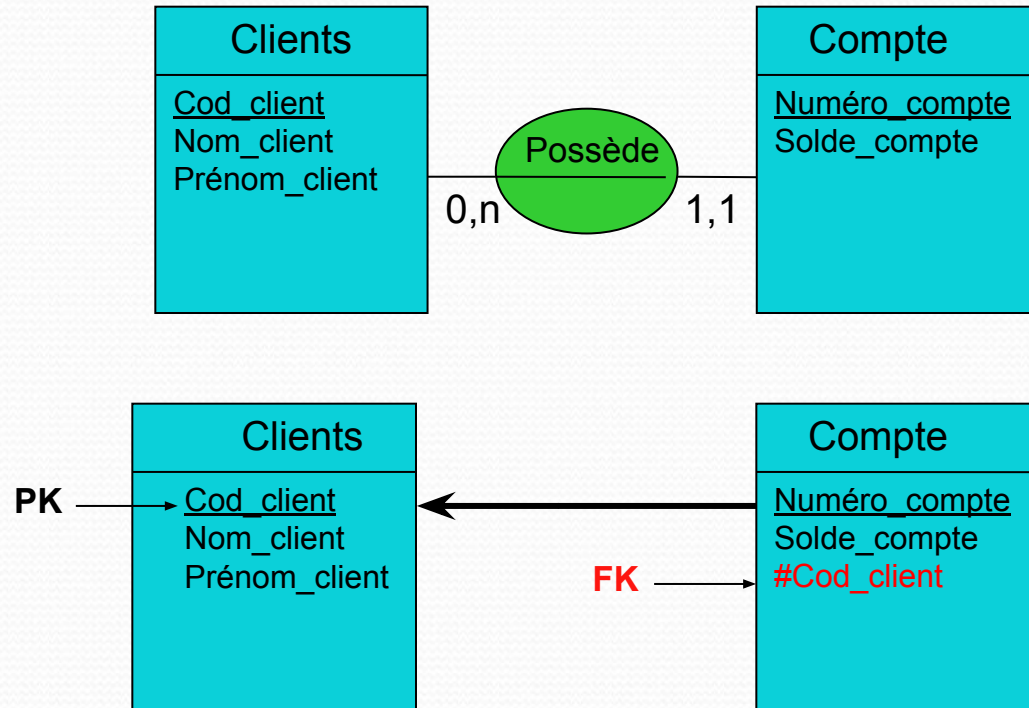


Relation de 1 à n

MCD



MLD

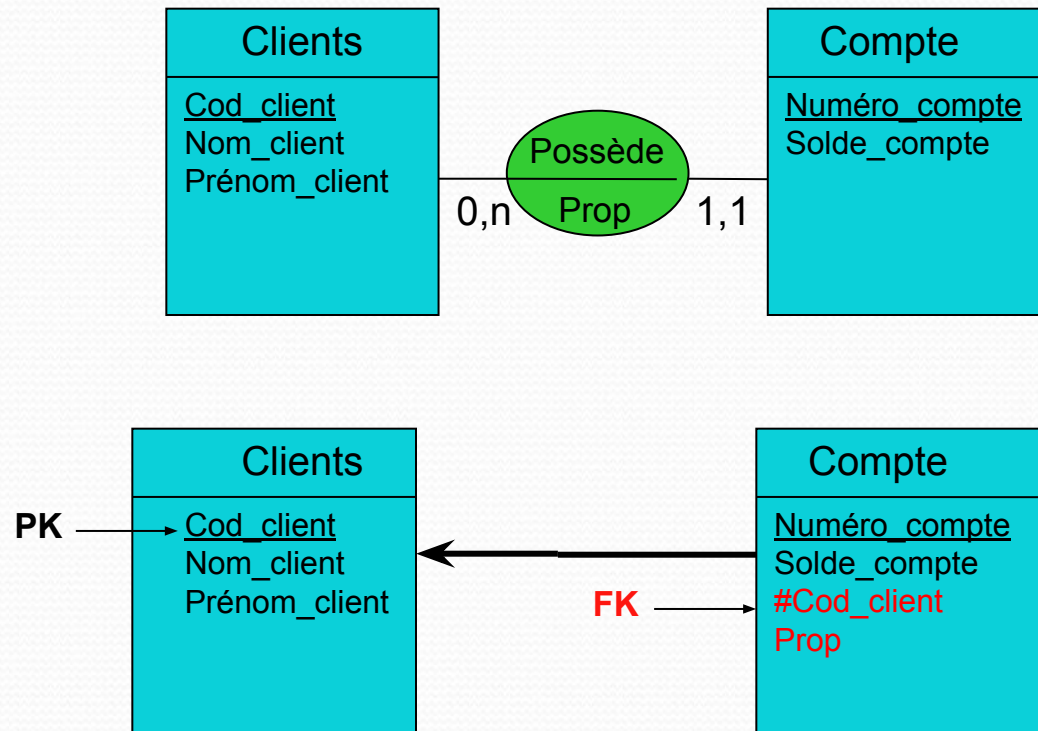


Relation de 1 à n

MCD



MLD

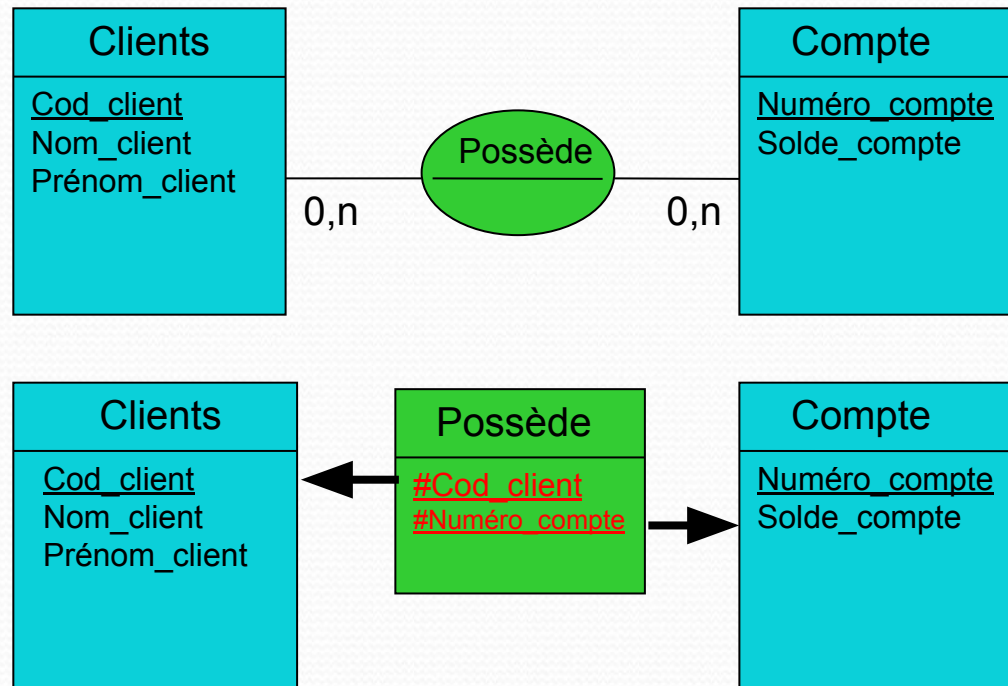


Liaison de n à n

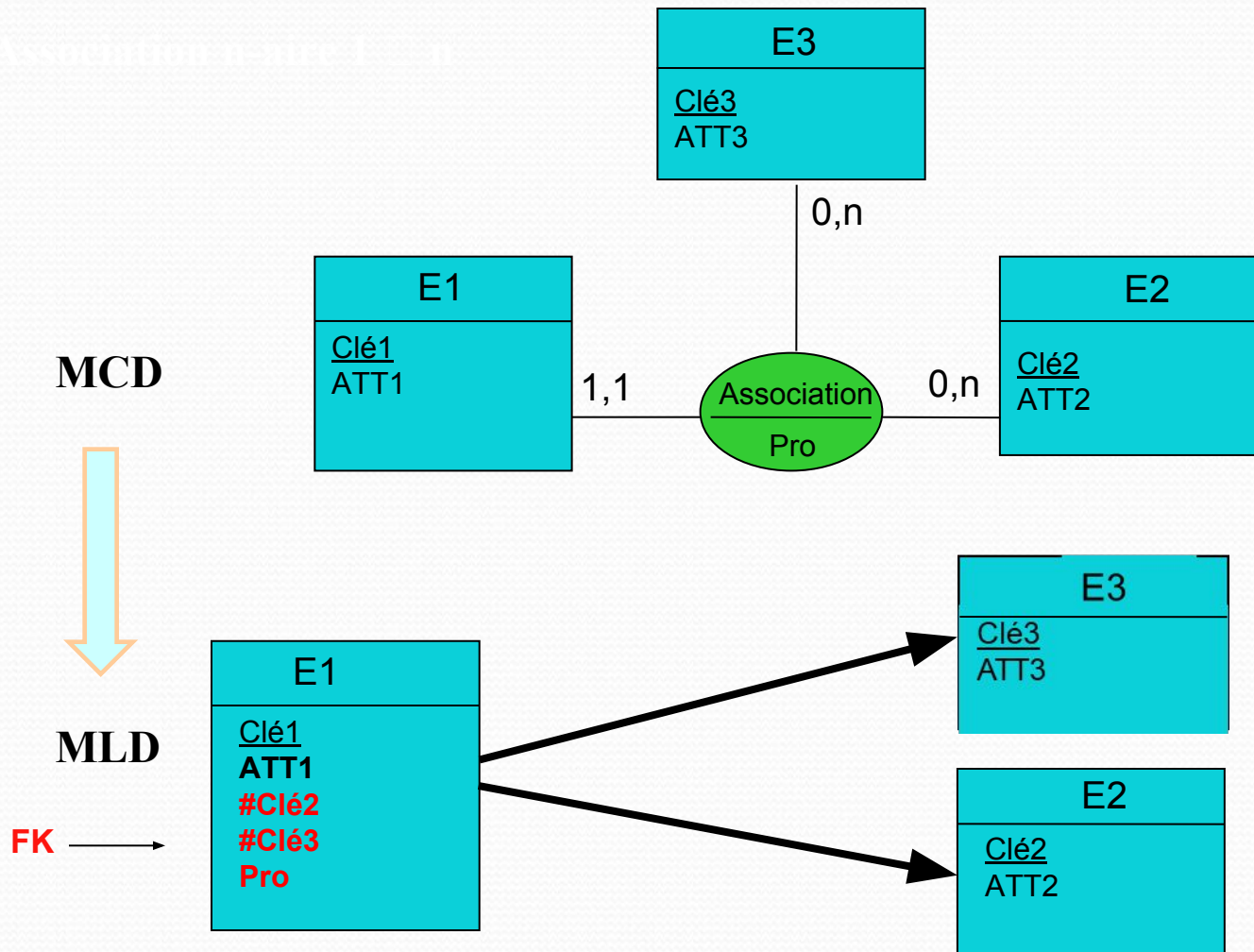
MCD



MLD



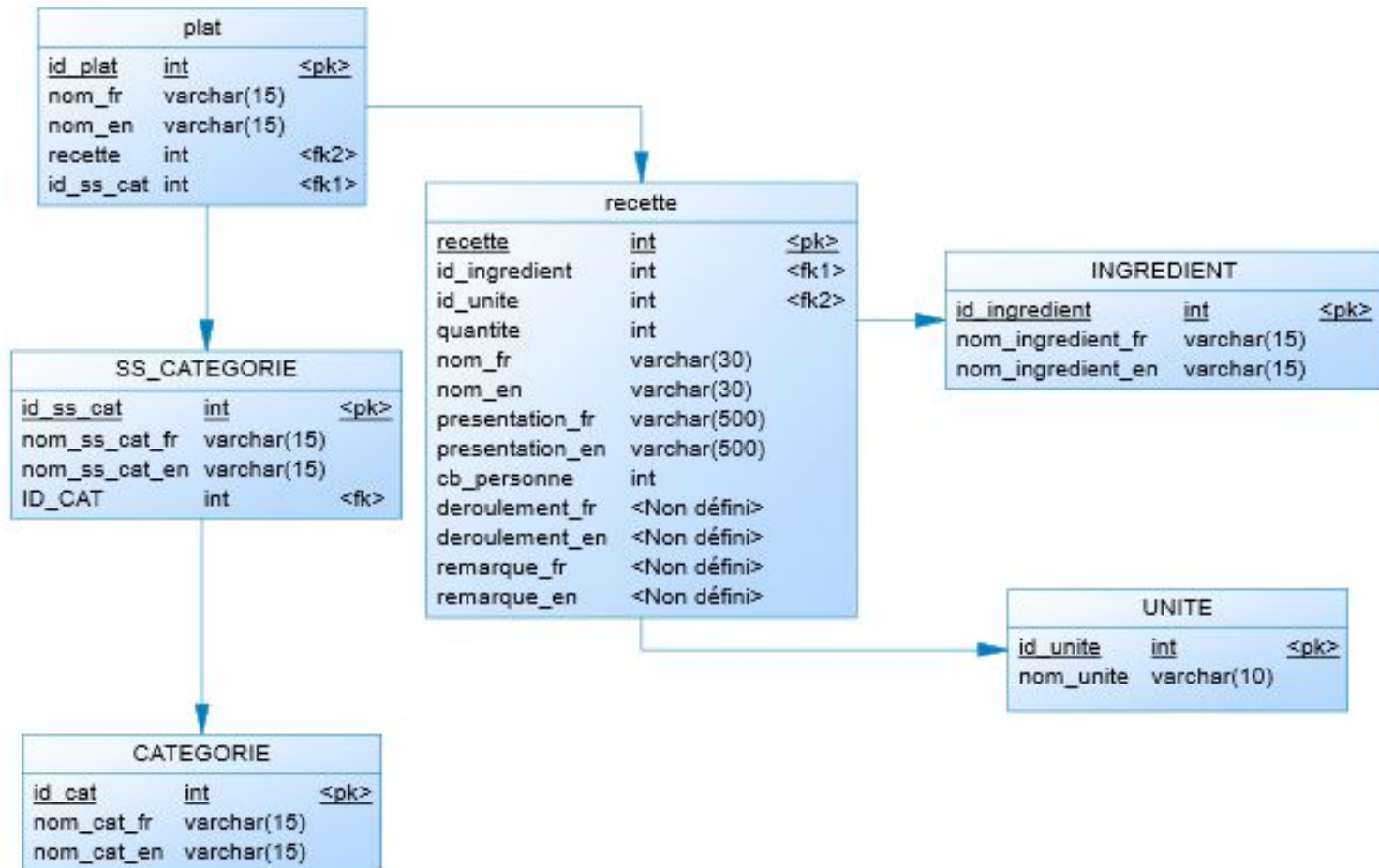
Association n-à-n





Passage du MLD à MPD

MPD





Création de la BDD

SQL

- Une fois les tables décidées, il faut les créer
 - Plusieurs logiciels (MySQL, PostGres, Access, Oracle...)
- Tous utilisent le même langage de requête : le SQL
- Tout d'abord une présentation de SQL

Langage de gestion de bases de données relationnelles (SQL)

SQL signifie Structured Query Language c'est-à-dire Langage d'interrogation structuré.

En fait SQL est un langage complet de gestion de bases de données relationnelles. Il a été conçu par IBM dans les années 70. Il est devenu le langage standard des systèmes de gestion de bases de données (SGBD) relationnelles (SGBDR).

Communiquer avec un SGBD au Moyen de SQL

**Saisie de
l'ordre SQL**

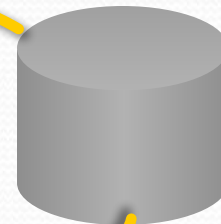
```
SQL> SELECT loc  
2 FROM dept;
```

**L'ordre est envoyé à
la base de données**

**Base de
données**

**Affichage des
données**

```
LOC  
-----  
NEW YORK  
DALLAS  
CHICAGO  
BOSTON
```



Création d'une BD

```
DROP DATABASE IF EXISTS db_ville;
```

```
CREATE DATABASE db_ville;
```

```
USE db_ville;
```

Créer une base de donnée :

```
/* Table des quartiers */  
CREATE TABLE quartiers (  
  id int NOT NULL auto_increment,  
  nom_quartier text,  
  PRIMARY KEY (id)  
);
```

```
/* Tables des lieux */  
CREATE TABLE lieux (  
  id int NOT NULL auto_increment,  
  nom text,  
  ref_quartier int,  
  description text,  
  PRIMARY KEY (id)  
);
```

Créer une base de donnée :

```
/* Table des quartiers */  
CREATE TABLE quartiers (  
  id int NOT NULL auto_increment,  
  nom_quartier text,  
  PRIMARY KEY (id)  
);
```

Nouveaux types...

```
/* Tables des lieux */  
CREATE TABLE lieux (  
  id int NOT NULL auto_increment,  
  nom text,  
  ref_quartier int,  
  description text,  
  PRIMARY KEY (id)  
);
```

Défini la clé principale

Insérer des valeurs

```
INSERT INTO quartiers VALUES  
  (1, 'Wazemmes');
```

```
INSERT INTO quartiers(nom_quartier)  
VALUES ('Vieux Lille');
```

```
INSERT INTO lieux VALUES  
  ('Biplan', 1, 'Salle de spectacle');
```

Insérer des valeurs

```
INSERT INTO quartiers VALUES  
  (1, 'Wazemmes');
```

```
INSERT INTO quartiers (nom quartier)  
VALUES ('Vieux Lille');
```

```
INSERT INTO lieux VALUES  
  ('Biplan', 1, 'Salle de spectacle');
```

Permet d'ajouter qu'une
partie d'un **tuple**

Le numéro de la clé est
sous entendu

Interrogation de la base

- Tout le contenu de la table quartiers :
 - `SELECT * FROM quartiers;`
- Tous les noms de lieux:
 - `SELECT nom FROM lieux;`
- Même chose mais classé par ordre alphabétique :
 - `SELECT nom FROM lieux ORDER BY nom;`
- Compter le nombre de lieux différents :
 - `SELECT COUNT(DISTINCT nom) FROM lieux;`

Interrogation suite

- Tous les noms de lieux qui se trouvent dans le quartier "Q1":
 - `SELECT lieux.nom FROM lieux,quartiers WHERE lieux.ref_quartier=quartiers.id and quartiers.nom="Q1";`
- Cette opération s'appelle une **jointure**
 - Elle se fait grâce à la clé de la table quartiers :
 - `ref_quartier=quartiers.id`

Modification

- UPDATE quartiers SET nom_quartier='Waazzeemmes' WHERE id=1;

Suppression

- DROP TABLE quartiers;
 - Supprime toute la table mais il faut qu'elle soit vide
- DELETE FROM lieux;
 - Vide toute la table
- DELETE FROM lieux WHERE ref_quartier=1;
 - Supprime toutes les lignes où ref_quartier=1



TD PHPMYADMIN